

DMGT Unit 1

Adesh Lokhande

1. Sets

Set ek group hota hai unique elements ka.

Isme repeat values nahi hoti, aur order important nahi hota.

Example:

$A = \{1, 2, 3\}$

Types of sets:

- **Empty set:** Jisme koi element nahi hota $\rightarrow \{\}$
- **Finite set:** Jisme elements count ho sakte hain
- **Infinite set:** Jisme elements endless hote hain
- **Subset:** Agar B ke saare elements A me ho, to B subset of A hai

Basic operations:

- **Union ($A \cup B$):** Dono sets ke saare unique elements
- **Intersection ($A \cap B$):** Dono me common elements
- **Difference ($A - B$):** A ke wo elements jo B me nahi hain

Simple real example:

Ek bag me {Apple, Mango, Banana} ho to ye ek set hai — bas ye dekhna hota hai kaunse fruits hain, quantity matter nahi karti.

2. Venn Diagram

Venn diagram circles ka use karke sets ke relations ko visually dikhata hai. Ye samajhne me help karta hai ki sets kaunse parts me overlap ho rahe hain aur kaunse alag hain.

Main uses:

- **Union:** Dono circles ka combined area
- **Intersection:** Circles ka overlapping area
- **Difference:** A circle ka wo hissa jo B me nahi aata
- **Complement:** Universal set ka wo part jo A ke bahar hota hai

Simple real example:

A = math pasand karne wale students

B = science pasand karne wale students

Venn diagram se pata chal jaata hai kaun sirf math pasand karta hai, kaun sirf science, aur kaun dono subjects pasand karta hai.

3. Cartesian Product

Cartesian product do sets ke elements ko pair form me combine karta hai.

Matlab A ka har element B ke har element ke saath pair banata hai.

Example: $A = \{1, 2\}$ $B = \{x, y\}$ $A \times B =$ $\{$
 $(1, x),$
 $(1, y),$
 $(2, x),$
 $(2, y)$
 $\}$

Isme order important hota hai, matlab $(1, x)$ aur $(x, 1)$ alag hote hain.

Simple real example:

Shirt colors = {Red, Blue}

Sizes = {M, L}

Possible combinations =

Red-M, Red-L, Blue-M, Blue-L

Yahi Cartesian product ka concept hai.

4. Power Set

Power set matlab kisi set ke saare possible subsets ka collection.

Isme empty set se lekar full set tak har combination include hota hai.

Example: $A = \{1, 2\}$ Power set $P(A) =$ $\{$
 $\{\},$
 $\{1\},$
 $\{2\},$
 $\{1, 2\}$
 $\}$

Agar set me n elements hain, to power set me 2^n subsets hote hain.

Simple real example:

Agar toppings = {Cheese, Corn}

To possible choices:

- No topping
- Sirf Cheese
- Sirf Corn
- Dono

Ye hi power set hota hai.

5. Cardinality and Countability

Cardinality set ka size hota hai, matlab usme kitne elements hain.

Example:

$A = \{3, 5, 7\}$

Cardinality = 3

Countability

Infinite sets do tarah ke hote hain:

1. **Countable Infinite:**

Jise hum list ya numbering se match kar sakte hain.

Example: Natural numbers $\{1, 2, 3, \dots\}$

2. **Uncountable Infinite:**

Jise list banana possible nahi hota.

Example: Real numbers (0 se 1 ke beech wale)

Simple idea:

Countable sets ko ek order me likh sakte ho,

Uncountable sets ko nahi.

6. Propositional Logic

Propositional logic statements ke truth (True/False) ke basis par kaam karta hai.

Har statement ek simple fact hota hai jiska result True ya False hota hai.

Examples of propositions:

- “2 even hai” $\rightarrow$ True
- “5 divisible by 4” $\rightarrow$ False

Isme hum statements ko p, q, r jaise symbols se represent karte hain, takki reasoning easy ho.

Use kaha hota hai:

- Decision making
- Proofs
- Computer logic circuits

Ye logic ka basic foundation hota hai.

7. Logical Connectives

Logical connectives woh tools hote hain jo do statements ko combine karke unka final True/False result nikalne me help karte hain.

Main connectives:

1. **NOT ($\neg p$)**

Statement ka opposite kar deta hai.

Agar $p = \text{True}$, to $\neg p = \text{False}$.

2. **AND ($p \wedge q$)**

Dono statements True hon tabhi result True hota hai.

3. **OR ($p \vee q$)**

Agar p ya q me se koi ek bhi True ho, to result True.

4. **IMPLIES ($p \rightarrow q$)**

Agar p True hai to q bhi True hona chahiye.

Sirf us case me False hota hai jab p True aur q False.

5. **BICONDITIONAL ($p \leftrightarrow q$)**

Jab p aur q dono same value ke ho (True-True ya False-False), tab True hota hai.

Simple example:

p: "Aaj holiday hai"

q: "Main movie dekhunga"

In dono ko connectives ke saath combine karke decisions banaye ja sakte hain.

8. Truth Tables

Truth table ek chart hota hai jisme hum statements aur unke connectives ka result har possible case me check karte hain.

Isse pata chalta hai ki final statement kab True hoga aur kab False.

Example:

p aur q dono statements ho sakte hain:

Possible combinations:

- $p = T, q = T$
- $p = T, q = F$
- $p = F, q = T$
- $p = F, q = F$

Har row me hum AND, OR, NOT, IMPLIES ka result nikal sakte hain.

Simple example ($p \text{ AND } q$):

- $T \text{ AND } T \rightarrow T$
- $T \text{ AND } F \rightarrow F$
- $F \text{ AND } T \rightarrow F$
- $F \text{ AND } F \rightarrow F$

Truth tables reasoning, circuits aur logic proofs ko exactly samajhne ka best tariqa hote hain.

9. Normal Forms

Normal forms logic statements ko ek fixed, standard structure me likhne ka tarika hota hai. Isse statements ko simplify karna aur compare karna easy ho jata hai.

Do main types hote hain:

1. Conjunctive Normal Form (CNF)

Statement AND ke combination ke form me hota hai, aur AND ke andar OR wale parts hote hain.

Example format:

$(p \vee q) \wedge (\neg r \vee s)$

2. Disjunctive Normal Form (DNF)

Statement OR ke combination me hota hai, aur OR ke andar AND wale parts hote hain.

Example format:

$$(p \wedge q) \vee (\neg r \wedge s)$$

Use:

- Logic simplification
- Boolean algebra
- Computer circuit design

Normal forms statements ko ek uniform pattern me convert karne ka easy method deta hai.

10. Validity

Validity ka matlab hota hai ki koi logical statement har possible situation me True ho.

Agar kisi expression ko jitni bhi values de do, wo kabhi False na bane, to wo **valid** hota hai.

Example:

Statement:

$$p \vee \neg p$$

("p ya NOT p")

Chahe p True ho ya False,

$p \vee \neg p$ hamesha True hi rahega.

Isliye ye valid statement hai.

Simple idea:

Valid statement = har condition me True.

Ye logic me ek universal truth jaisa hota hai.

11. Predicate Logic

Predicate logic propositional logic se zyada powerful hota hai, kyunki ye statements ke andar objects, properties aur relations ko bhi handle karta hai.

Isme hum **predicates** use karte hain, jo kisi object ke baare me kuch batate hain.

Example:

$$P(x) = \text{"x even hai"}$$

Agar $x = 4$, to $P(4)$ True

Agar $x = 5$, to $P(5)$ False

Predicate logic variable-based reasoning allow karta hai, jaise x, y, z.

Isse hum real-life jaise statements ko clearly express kar sakte hain.

Real example:

$$L(x) = \text{"x laptop use karta hai"}$$

Isme x koi bhi person ho sakta hai.

Predicate logic ka use mathematics, programming aur AI reasoning me bahut hota hai.

12. Limitations of Predicate Logic

Predicate logic powerful hai, lekin har real-world situation ko perfectly express nahi kar sakta. Kuch cheezein iske scope se bahar reh jaati hain.

Main limitations:

1. **Complex real-world meanings handle nahi kar sakta**
Emotions, time-based changes, ya human intentions ko clearly express nahi kar pata.
2. **Uncertain situations represent karna mushkil**
Jaise “shayad”, “zyaadatar”, “kabhi kabhi” jaisi statements predicate logic me directly express nahi hoti.
3. **Higher-order reasoning missing**
Predicate logic me hum variables ko objects ke liye use karte hain,
lekin predicates ke upar predicates apply nahi kar sakte (ye higher-order logic me aata hai).
4. **Context ka issue**
Same sentence ka meaning situation ke hisaab se badal jata hai, lekin predicate logic fix structure follow karta hai.

Simple idea:

Predicate logic strong hai, par real-life ki har nuance ko exactly capture nahi kar pata.

13. Universal Quantification

Universal quantification ka matlab hota hai **“har ek ke liye”**.

Isko symbol $\forall$ se represent karte hain.

Statement ka structure:

$\forall x P(x)$

Matlab: “Sabhi x ke liye P(x) true hai.”

Example:

$\forall x (x + 0 = x)$

Ye math me har number ke liye true hota hai.

Simple real-life example:

$\forall x$: “Har student ko ID card lana zaroori hai.”

Matlab ek bhi student aisa nahi hona chahiye jo is rule se bahar ho.

Universal quantification Python me “for all items in list” logic jaisa hota hai.

14. Existential Quantification

Existential quantification ka matlab hota hai **“koi ek exist karta hai”** ya **“at least ek”**.

Isko symbol $\exists$ se show kiya jata hai.

Structure:

$\exists x P(x)$

Matlab: “Koi ek x aisa hai jiske liye P(x) true hai.”

Example:

$\exists x (x \text{ is even})$

Matlab kam se kam ek number even hai.

Ye statement True hai kyunki 2, 4, 6... sab even hain.

Simple real-life example:

$\exists x$: “Class me kam se kam ek student glasses pehenta hai.”

Matlab poori class me se sirf ek student bhi ho to statement true ho jata hai.

Existential quantifier “at least one” condition ko represent karta hai.

15. First Order Logic (FOL)

First Order Logic, predicate logic ka extended version hota hai.

Isme **objects**, **properties**, **relations**, aur **quantifiers** ka use hota hai, jisse hum real-world statements ko clearly express kar sakte hain.

FOL ke main parts:

- **Variables:** x, y, z
- **Predicates:** $P(x), Q(x, y)$
- **Quantifiers:** $\forall$ (for all), $\exists$ (there exists)
- **Connectives:** AND, OR, NOT, IMPLIES
- **Functions:** $f(x), g(x, y)$

Example:

$\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$

Meaning: “Har human mortal hota hai.”

FOL ka use:

- Mathematics proofs
- Databases
- AI reasoning
- Knowledge representation

First Order Logic propositional logic se zyada expressive hota hai, kyunki ye objects aur unke relations ko detail me describe kar sakta hai.

16. Mathematical Induction

Mathematical induction ek method hai jisse hum prove karte hain ki koi statement **sab natural numbers** ke liye true hai.

Isme do steps hote hain:

1. Base Case:

Sabse chhote number (usually $n = 1$) ke liye statement true dikhana.

2. Inductive Step:

Assume karo ki statement $n = k$ ke liye true hai.

Phir prove karo ki $n = k + 1$ ke liye bhi true hona hi chahiye.

Agar dono steps complete ho jaaye, to statement har natural number ke liye true maan liya jata hai.

Simple example:

$$1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Isko proof karne ke liye induction ka use hota hai.

Mathematical induction ek chain reaction jaisa hota hai—
pehla link (base case) sahi ho aur har link next ko pull kare (inductive step),
to poori chain valid ho jati hai.

17. The Well-Ordering Principle

Well-Ordering Principle kehta hai ki **natural numbers ka har non-empty set ek smallest element rakhta hai.**

Matlab agar tum kisi bhi group of natural numbers ko lo,
chahe wo random ho, unordered ho, ya infinite ho,
usme hamesha ek number sabse chhota milega.

Examples:

- Set = {5, 9, 12} → smallest = 5
- Set = {20, 3, 100, 7} → smallest = 3
- Set = {10, 11, 12, ...} → smallest = 10

Important point:

Ye rule sirf **natural numbers** par apply hota hai.
Integers par nahi, kyunki integers me lower bound hi nahi hota.

Simple real example:

Agar tumhare paas kuch bills ki list ho {120, 450, 78, 330},
to tum hamesha likh sakte ho ki sabse kam amount 78 hai.

18. Recursive Definition

Recursive definition kisi cheez ko define karne ka tarika hai jisme
current value ko previous value ke base par define kiya jata hai.

Isme do cheezein hoti hain:

1. **Base Case:**
Starting value jiske bina recursion start nahi ho sakta.
2. **Recursive Step:**
Rule jo batata hai ki agla element kaise milega.

Common example: Fibonacci series

- Base: $F(0) = 0$, $F(1) = 1$
- Recursive: $F(n) = F(n-1) + F(n-2)$

Another example: Factorial

- Base: $0! = 1$
- Recursive: $n! = n \times (n-1)!$

Simple idea:

Recursive definition ek chain banata hai jisme har value pichhli value par depend karti hai.

19. The Division Algorithm

Division algorithm ek simple rule batata hai ki
koi bhi integer **a** ko kisi positive integer **b** se divide karne par
humesha do numbers milte hain:

- **q → quotient**
- **r → remainder**

Aur relation hamesha ye hota hai:

$$\mathbf{a = bq + r}$$

$$\text{jaha } \mathbf{0 \leq r < b}$$

Example:

$$a = 23, b = 5$$

$$23 = 5 \times 4 + 3$$

$$\text{yaha } q = 4$$

$$r = 3$$

Simple real example:

Agar tumhe 23 chocolates ko 5 ke group me baantna ho,
to 4 groups banenge
aur 3 chocolates bachi rahengi.

20. Prime Numbers

Prime number wo natural number hota hai jo sirf **2 divisors** rakhta hai:
1 aur khud.

Examples:

2, 3, 5, 7, 11, 13 ...

Important points:

- 2 duniya ka **pehla aur eklauta even prime number** hai.
- 1 prime nahi hota, kyunki uska sirf 1 divisor hota hai.
- Prime numbers building blocks hote hain saare natural numbers ke.

Simple real example:

Jaise bricks se building banti hai,
waise hi prime numbers se saare numbers bante hain (factorization ke through).

21. The Greatest Common Divisor (GCD)

GCD do numbers ka **sabse bada common divisor** hota hai.
Matlab wo largest number jo dono numbers ko completely divide kar de,
remainder zero aaye.

Example:

12 aur 18 ke divisors:

- $12 \rightarrow 1, 2, 3, 4, 6, 12$
- $18 \rightarrow 1, 2, 3, 6, 9, 18$

Common divisors = 1, 2, 3, 6

To GCD = **6**

Simple real example:

Agar tum 12 aur 18 chocolates ko equal groups me divide karna chaho, aur dono me groups ka size same rakho, to maximum group size 6 ho sakta hai. Yahi GCD ka concept hai.

22. Euclidean Algorithm

Euclidean Algorithm GCD nikalne ka fast method hai.

Iska idea simple hai:

Bade number ko chhote number se repeatedly divide karo, aur remainder ko next step me use karo, jab tak remainder 0 na ho. Last non-zero remainder hi GCD hota hai.

Example: GCD(48, 18)

1. $48 \div 18 = 2$ remainder 12
2. $18 \div 12 = 1$ remainder 6
3. $12 \div 6 = 2$ remainder 0

Last non-zero remainder = **6**

So GCD = **6**

Simple real idea:

Bade number ko chhote se divide karte jao, bacha hua remainder se process repeat karo. Ye method hamesha quickly GCD de deta hai.

23. The Fundamental Theorem of Arithmetic

Fundamental Theorem of Arithmetic kehta hai ki:

Har integer (1 se bada) ko prime numbers ke product ke form me uniquely likha ja sakta hai.

Iska matlab:

Har number ka **unique prime factorization** hota hai.

Order change ho sakta hai, but factors same rahenge.

Examples:

- $30 = 2 \times 3 \times 5$
- $60 = 2 \times 2 \times 3 \times 5$
- $84 = 2 \times 2 \times 3 \times 7$

Chahe kitni baar alag tareeke se factorization try karo, prime factors hamesha yehi rahenge.

Simple real idea:

Prime numbers building blocks hote hain.

Jaise har building bricks se banti hai,

waise hi har number prime numbers se banta hai.